

Testes

```
#include <Unit/CUnit.h>
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include "../include/estruturas.h"
#include "../include/logica.h"
#include "../include/reutilizado.h"
#include "../include/dsl.h"

//
// Testes de cartas e vitória
//

// Função que testa se o baralho é criado com sucesso
void test_criar_baralho(void) {
    Carta *deck = criarBaralho(1);
    CU_ASSERT_PTR_NOT_NULL(deck); // Testa se consegue criar o baralho
    CU_ASSERT_EQUAL(deck[0].num, 1);
    CU_ASSERT_EQUAL(deck[13].naipe, 1); // Se a primeira carta é o Ás de Espadas
    free(deck);
}

// Função que testa se quando as cartas são baralhadas não ocorrem erros
void test_baralhar_cartas(void) {
    Carta *deck = criarBaralho(1); // cria um baralho
    baralharCartas(deck, 52); // Baralha as cartas
    CU_ASSERT_PTR_NOT_NULL(deck); // Verifica se o apontador não é nulo
    free(deck);
}

// Teste que verifica se a vitória é corretamente detetada
void test_verificar_vitoria(void) {
    EstadoJogo jogo; Pilha p1;
    jogo.total_pilhas = 1; jogo.pilhas = &p1; // cria um jogo com 1 pilha
    strcpy(p1.nome_tipo, "TAB");
    p1.quantidade = 3;
    CU_ASSERT_EQUAL(verificar_vitoria(&jogo), 0); // Com 3 cartas não pode haver vitória
    p1.quantidade = 0;
    CU_ASSERT_EQUAL(verificar_vitoria(&jogo), 1); // tem de ter vitória
}

//
// Testes para regras básicas do jogo
//

// Função que testa as regras de sequencia de cor
void test_regra_cor_sequencia(void) {
    Carta c4 = {4, 1};
    Carta c5 = {5, 1};
    Carta c5_preto = {5, 0};
    CU_ASSERT_EQUAL(regra_sequencia(c4, c5, '<'), 1);
    CU_ASSERT_EQUAL(regra_sequencia(c4, c5, '>'), 0); // Se for decrescente tem de dar 0
    CU_ASSERT_EQUAL(regra_cor(c4, c5), 0); // tem cores iguais
    CU_ASSERT_EQUAL(regra_cor(c4, c5_preto), 1); // tem cor diferentes
}
```


// Função que testa a implementação das regras no jogo

void test_aplicar_regras(void) {

Carta co = {2, 1}; Carta cd = {3, 0};

CU_ASSERT_EQUAL(aplicar_regras(co, cd, "D<"), 1);

CU_ASSERT_EQUAL(aplicar_regras(co, cd, "M<"), 0);

}

// Função auxiliar para preparar dados

void aux_prep_regras(EstadoJogo *e, Regra *r, Pilha p[]) {

strcpy(p[0].nome_tipo, "T1"); p[0].quantidade = 1;

strcpy(p[1].nome_tipo, "T2"); p[1].quantidade = 1;

strcpy(p[2].nome_tipo, "L"); p[2].quantidade = 1;

strcpy(r->origem, "T1"); strcpy(r->destino, "T2"); strcpy(r->flags, "D<");

e->total_regras = 1; e->regras = r;

}

// Função que testa se a regra é encontrada

void test_encontrar_regra(void) {

EstadoJogo e; Regra r1; Pilha p[3];

aux_prep_regras(&e, &r1, p);

CU_ASSERT_STRING_EQUAL(encontrar_regra(&e, &p[0], &p[1]), "D<");

CU_ASSERT_PTR_NULL(encontrar_regra(&e, &p[0], &p[2]));

}

// Função que testa se o computador encontra as regras de automatico

void test_tratar_auto(void) {

EstadoJogo e;

e.total_auto_regras = 0;

e.auto_regras = NULL;

tratar_auto(&e, "TAB", "FUND", "AV");

CU_ASSERT_EQUAL(e.total_auto_regras, 1);

CU_ASSERT_STRING_EQUAL(e.auto_regras[0].origem, "TAB");

CU_ASSERT_STRING_EQUAL(e.auto_regras[0].destino, "FUND");

CU_ASSERT_STRING_EQUAL(e.auto_regras[0].flags, "AV");

free(e.auto_regras);

}

// Função que testa se os movimentos automaticos estão a ser feitos de maneira correta

void test_verificar_vazio(void) {

Carta as = {1, 0};

Carta rei = {13, 0};

Carta normal = {5, 0};

CU_ASSERT_EQUAL(verificar_vazio(as, "AV"), 1);

CU_ASSERT_EQUAL(verificar_vazio(rei, "AV"), 0);

CU_ASSERT_EQUAL(verificar_vazio(rei, "KV"), 1);

CU_ASSERT_EQUAL(verificar_vazio(normal, "K"), 0);

CU_ASSERT_EQUAL(verificar_vazio(normal, "V"), 1);

}

// Função que verifica se a regra 'V' bate certo com o estado de espaço ocupado/vazio

void test_regra_combina_estado(void) {

Testa se a regra 'V' bate certo com o estado de espaço ocupado/vazio quando as regras permitem ou não

teste para colocar um As no fundo de um determinado estado


```

    CU_ASSERT_EQUAL(regra_combina_estado(0, "AV", 1);
    CU_ASSERT_EQUAL(regra_combina_estado(1, "AV", 0);
    CU_ASSERT_EQUAL(regra_combina_estado(1, "D<", 1);
    CU_ASSERT_EQUAL(regra_combina_estado(0, "D<", 0);
}

// Função que verifica se o sistema reconhece um bloco de cartas perfeito
void test_verificar_bloco(void) {
    Pilha p; Carta c[3];
    c[0].num = 5; c[0].naipe = 1;
    c[1].num = 4; c[1].naipe = 1;
    c[2].num = 3; c[2].naipe = 1; } → Bloco ordenado
    p.cartas = c; p.quantidade = 3;
    CU_ASSERT_EQUAL(verificar_bloco(&p, 3), 1);
    c[1].naipe = 2;
    CU_ASSERT_EQUAL(verificar_bloco(&p, 3), 0);
}

// Função que verifica as novas barreiras de segurança para espaços vazios
void test_decidir_vazio(void) {
    Carta as = {1, 0}, rei = {13, 0}, normal = {5, 0};
    Pilha fund, tab;
    strcpy(fund.nome_tipo, "FUND");
    strcpy(tab.nome_tipo, "TAB");
    CU_ASSERT_EQUAL(decidir_vazio(as, &fund, "AV", 1), 1);
    CU_ASSERT_EQUAL(decidir_vazio(normal, &fund, "AV", 1), 0);
    CU_ASSERT_EQUAL(decidir_vazio(rei, &tab, "KV", 1), 1);
    CU_ASSERT_EQUAL(decidir_vazio(normal, &tab, "0", 1), 1);
}

//
// Teste de movimentos
//

// Função auxiliar para criar cartas e pilhas que vão ser usadas nos testes
// de movimento
void preparar_cartas_e_pilhas(Pilha *p, Carta *co, Carta *cd) {
    memset(p, 0, 2 * sizeof(Pilha));
    co->num = 2; co->naipe = 1;
    cd->num = 3; cd->naipe = 0;
    strcpy(p[0].nome_tipo, "P1");
    p[0].quantidade = 1; p[0].cartas = co;
    strcpy(p[1].nome_tipo, "P2");
    p[1].quantidade = 1; p[1].cartas = cd;
}

// Função auxiliar que cria regras e o estado para ser usado nos testes de
// movimento
void preparar_regras_e_estado(EstadoJogo *e, Pilha *p, Regra *r) {
    strcpy(r->origem, "P1");
    strcpy(r->destino, "P2");
    strcpy(r->flags, "D<");
    e->total_pilhas = 2; e->pilhas = p;
    e->total_regras = 1; e->regras = r;
}

```

Handwritten notes:

- ← tem de dar
- ← não vazia
- ← coluna com cartas mas que aceita
- ← coluna vazia
- ← tem de dar verdadeiro
- ← não está
- ← Para jogar com AV tem de decidir o AS
- ← Normal não pode
- ← tem de conseguir
- ↑ Permite que qualquer carta possa ir
- ← Limpa lixo de memória
- ← Cria cartas e pilhas para usar nos testes de movimento
- ← Cria regras para o estado

// Função que testa um movimento entre duas cartas de cor diferente

```
void test_movimento_normal(void) {  
    Pilha p[2]; Carta co, cd;  
    preparar_cartas_e_pilhas(p, &co, &cd);  
    CU_ASSERT_EQUAL(verificar_movimento(&p[0], &p[1], "D<", 1), 1);  
}
```

Como não é um
Co, tem de dar 1
tem de ser di.

// Função que testa o movimento para uma pilha vazia quando as regras não o permitem

```
void test_mov_vazio_falso(void) {  
    Pilha p[2]; Carta co, cd;  
    preparar_cartas_e_pilhas(p, &co, &cd);  
    p[1].quantidade = 0;  
    CU_ASSERT_EQUAL(verificar_movimento(&p[0], &p[1], "K", 1), 0);  
}
```

para colocar uma
carta que não é
Rei sendo que a
regra o exige logo
tem de dar 0

// Função que testa o movimento para uma pilha vazia quando as regras permitem

```
void test_mov_vazio_verdadeiro(void) {  
    Pilha p[2]; Carta co, cd;  
    preparar_cartas_e_pilhas(p, &co, &cd);  
    p[1].quantidade = 0;  
    CU_ASSERT_EQUAL(verificar_movimento(&p[0], &p[1], "0", 1), 1);  
}
```

permite movimento para vazio

// Função que testa se a função testar_destinos consegue encontrar um destino válido num cenário possível

```
void test_testar_destinos(void) {  
    EstadoJogo e; Pilha p[2]; Regra r; Carta co, cd;  
    preparar_cartas_e_pilhas(p, &co, &cd);  
    preparar_regras_e_estado(&e, p, &r);  
    int d_o, d_d;  
    CU_ASSERT_EQUAL(testar_destinos(&e, 0, &d_o, &d_d), 1);  
}
```

tem de conseguir encontrar um sitio para mover a carta

// Função que testa se conseguiu pedir dica

```
void test_pedir_dica(void) {  
    EstadoJogo e; Pilha p[2]; Regra r; Carta co, cd;  
    preparar_cartas_e_pilhas(p, &co, &cd);  
    preparar_regras_e_estado(&e, p, &r);  
    int d_o, d_d;  
    CU_ASSERT_EQUAL(pedir_dica(&e, &d_o, &d_d), 1);  
}
```

Toda se consegue pedir a dica

// Função que testa o movimento de cartas entre duas pilhas

```
void test_executar_movimento_real(void) {  
    Pilha o, d; o.quantidade = 2; d.quantidade = 0;  
    o.cartas = malloc(2 * sizeof(Carta)); d.cartas = NULL;  
    executar_movimento_real(&o, &d, 1);  
    CU_ASSERT_EQUAL(o.quantidade, 1);  
    CU_ASSERT_EQUAL(d.quantidade, 1);  
    CU_ASSERT_PTR_NOT_NULL(d.cartas);  
    free(o.cartas); free(d.cartas);  
}
```

Verifica se tudo foi feito corretamente


```

//
// Testes da funcionalidade UNDO
//

// Função que testa se é realizada a cópia da pilha de maneira correta
void test_copiar_pilha(void) {
    Pilha o, d;
    strcpy(o.nome_tipo, "TAB"); o.quantidade = 1;
    o.cartas = malloc(sizeof(Carta));
    copiar_pilha(&d, &o); // copia uma pilha
    CU_ASSERT_STRING_EQUAL(d.nome_tipo, "TAB");
    CU_ASSERT_EQUAL(d.quantidade, 1);
    CU_ASSERT_PTR_NOT_NULL(d.cartas);
    free(o.cartas); free(d.cartas); // libera
}

// Função que testa se o UNDO é feito corretamente
void test_fluxo_undo(void) {
    EstadoJogo *e1 = malloc(sizeof(EstadoJogo));
    e1->total_pilhas = 0; e1->total_regras = 0; e1->anterior = NULL;
    EstadoJogo *e2 = guardar_estado(e1); // guarda o estado
    CU_ASSERT_PTR_NOT_NULL(e2); // testa se o e2 aponta para e1
    CU_ASSERT_PTR_EQUAL(e2->anterior, e1); // testa se o ponteiro é para e1
    EstadoJogo *e_revertido = fazer_undo(e2); // faz undo
    CU_ASSERT_PTR_EQUAL(e_revertido, e1); // o ponteiro tem de ser e1 pois voltou a base
    free(e1);
}

//
// Motor do CUNIT
//

// Primeira parte dos testes
void adicionarTestes_parte1(CU_pSuite suite) {
    CU_add_test(suite, "test_criar_baralho", test_criar_baralho);
    CU_add_test(suite, "test_baralhar_cartas", test_baralhar_cartas);
    CU_add_test(suite, "test_verificar_vitoria", test_verificar_vitoria);
    CU_add_test(suite, "test_regra_cor_sequencia", test_regra_cor_sequencia);
    CU_add_test(suite, "test_aplicar_regras", test_aplicar_regras);
    CU_add_test(suite, "test_encontrar_regra", test_encontrar_regra);
    CU_add_test(suite, "test_movimento_normal", test_movimento_normal);
    CU_add_test(suite, "test_mov_vazio_falso", test_mov_vazio_falso);
}

// Segunda parte dos testes
void adicionarTestes_parte2(CU_pSuite suite) {
    CU_add_test(suite, "test_mov_vazio_verdadeiro", test_mov_vazio_verdadeiro);
    CU_add_test(suite, "test_testar_destinos", test_testar_destinos);
    CU_add_test(suite, "test_pedir_dica", test_pedir_dica);
    CU_add_test(suite, "test_executar_movimento_real", test_executar_movimento_real);
    CU_add_test(suite, "test_copiar_pilha", test_copiar_pilha);
}

```

Verifica se ficou tudo direito
libera
Entra o tabuleiro
guarda o estado
testa se o e2 aponta para e1
testa se o ponteiro é para e1
faz undo
o ponteiro tem de ser e1 pois voltou a base


```
CU_add_test(suite, "test_fluxo_undo", test_fluxo_undo);
CU_add_test(suite, "test_tratar_auto", test_tratar_auto);
CU_add_test(suite, "test_verificar_vazio", test_verificar_vazio);
}
```

// Terceira parte dos testes

```
void adicionarTestes_parte3(CU_pSuite suite) {
    CU_add_test(suite, "test_regra_combina_estado",
        test_regra_combina_estado);
    CU_add_test(suite, "test_verificar_bloco", test_verificar_bloco);
    CU_add_test(suite, "test_decidir_vazio", test_decidir_vazio);
}
```

// Função que configura a suite *resolva o ponteiro*

```
int configurarSuite(void) {
    CU_pSuite suite = CU_add_suite("Suite_Principal", NULL, NULL);
    if (NULL == suite) { se não foi criado
        CU_cleanup_registry(); limpa o registro
        return 0;
    }
```

Não tem nenhum handler porque não vamos usar os testes e não usamos um ficheiro externo com os testes

```
    adicionarTestes_parte1(suite);
    adicionarTestes_parte2(suite);
    adicionarTestes_parte3(suite);
    return 1;
}
```

Adiciona os testes

// Função que verifica se há testes a falhar

```
int verificarSeFalhou(CU_pTest pTest) {
    int falhou = 0; se não falhou
    CU_pFailureRecord pFailure = CU_get_failure_list();
    while (pFailure != NULL && falhou == 0) { Enquanto houver erros e não o encontrarmos
        if ((*pFailure).pTest == pTest) falhou = 1; se o endereço atual for igual ao que não passou, encontramos
        pFailure = (*pFailure).pNext;
    }
    return falhou;
}
```

Buena para o 1º ERRO

Se não avança

// Função auxiliar que imprime se o teste passou ou não

```
void imprimirTestesDaSuite(CU_pSuite pSuite) {
    CU_pTest pTest = (*pSuite).pTest; Pegamos no 1º teste
    while (pTest != NULL) { Enquanto houver testes
        if (verificarSeFalhou(pTest) == 1) se não foi passado
            printf("Funcao %-35s : \033[31mNAO PASSOU\033[0m\n",
                (*pTest).pName);
        else
            printf("Funcao %-35s : \033[32mPASSOU\033[0m\n",
                (*pTest).pName);
        pTest = (*pTest).pNext;
    }
```

Avança para o próximo teste

// Função para imprimir os resultados dos testes

```
void imprimirResultados(void) {
    printf("\n===== \n");
    printf("RESULTADO DOS TESTES \n");
}
```

```

printf("=====\n");
CU_pSuite pSuite = (*CU_get_registry()).pSuite; // Vão ao registro e pega o registro
while (pSuite != NULL) { // Enquanto houver testes
    imprimirTestesDaSuite(pSuite); // Imprime o resultado
    pSuite = (*pSuite).pNext; // Avança no teste
} // fizemos assim para seguir a arquitetura do CUnit que nos guarda os resultados
printf("=====\n\n");
// em arrays e nós podemos fazer mais 2; ele guarda um ponteiro que aponta
// para o próximo teste
}

```

```

//
// Main
//

```

```

int main(void) { // Se não conseguir inicializar dá erro
    if (CUE_SUCCESS != CU_initialize_registry()) return CU_get_error();
    if (configurarSuite() == 0) return CU_get_error(); // Se não conseguir configurar
    CU_run_all_tests(); // testa
    imprimirResultados(); // imprime
    CU_cleanup_registry(); // limpa o registro
    return CU_get_error();
} // Retorna erro caso exista

```